

# CS & IT ENGINEERING



## Computer Network

### Transport Layer

**Lecture No. - 12**



**By - Abhishek Sir**



# Recap of Previous Lecture



Topic

TCP Window Scaling

Topic

TCP Timers





# Topics to be Covered



Topic

Socket Programming

[Network Prog.]

# ABOUT ME



Hello, I'm **Abhishek**

- GATE CS AIR - 96
- M.Tech (CS) - IIT Kharagpur
- 12 years of GATE CS teaching experience

Telegram Link : [https://t.me/abhisheksirCS\\_PW](https://t.me/abhisheksirCS_PW)





## Topic : Sockets



- Socket is data structure inside the program
- Both client and server programs communicate via a pair of sockets

### Socket API :

- Set of library function calls
- Between Application Layer and Transport Layer
- Door between application process and end-to-end transport protocol





# Topic : Socket Programming



→ Client / server application that communicate using sockets

Process : Controlled by application developer

OSI Model : Mostly controlled by operating system



## Topic : Types of Sockets



Stream Socket : `[SOCK_STREAM]`

- Uses TCP protocol
- Connection-oriented service
- Reliable and byte stream oriented

Datagram Socket : `[SOCK_DGRAM]`

- Uses UDP protocol
- Connection-less service
- Unreliable datagram





## Topic : Creating a Socket



```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int socket ( int domain, int type, int protocol );
```

domain : Protocol Family  
[PF\_INET, AF\_INET, PF\_UNIX etc]

type : Communication Protocol semantics  
[SOCK\_STREAM, SOCK\_DGRAM]

protocol : Specify particular protocol, set "zero" as default

return : On success, a file descriptor for the new socket  
On error, -1 is returned

sockfd  
= socket ( );

→ Both TCP  
clients and  
server create  
socket

(sock\_fd)





## Topic : Bind a name to a Socket

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int bind ( int sockfd, const struct sockaddr *addr, socklen_t addrlen );
```

sockfd : file descriptor of the socket

return : On success, zero  
On error, -1 is returned

→ Bind : Optional for TCP client.



## Topic : Listen for connections



```
#include <sys/socket.h>
```

```
int listen ( int sockfd, int backlog );
```

sockfd : file descriptor of the socket at server TCP

backlog : maximum length for queue of pending connections

return : On success, zero  
On error, -1 is returned

→ Only by TCP server  
→ TCP server is ready to listen client req.





## Topic : Socket Programming with TCP



P<sub>2</sub>

TCP Server

- ① socket()
  - ② bind()
  - ③ listen()
- socket (Active)
- passive

#Q. Which one of the following socket API functions converts an unconnected active TCP socket into a passive socket?

[GATE-2014, Set-2, 1-Mark]

IIT-KGP

(A) connect

☒ (B) bind

☒ (C) listen

(D) ~~connect~~ accept

Ans: C





## Topic : Initiate a connection

→ only by TCP client 

```
#include <sys/socket.h>
```

```
int connect ( int sockfd, const struct sockaddr *addr, socklen_t addrlen );
```

sockfd : file descriptor of the socket at client TCP

return : If connection succeeds, zero  
On error, -1 is returned



# Topic : Socket Programming with TCP



$P_1$

TCP Client

① `socket()`

② `connect()`

`SYN` (Conn. Req.)

$P_2$

TCP Server

`socket()`

`bind()`

`listen()`



#Q. Which of the following system calls results in the sending of SYN packets?

[GATE-2008]

11SC

~~(A) socket~~

~~(B) bind~~

~~(C) listen~~

✓ (D) connect

Ans: D



## Topic : Accept a connection

→ Creates a new connected (connection) socket ✓

```
#include <sys/socket.h>
```

```
int accept ( int sockfd, struct sockaddr *_Nullable restrict addr,  
             socklen_t *_Nullable restrict addrlen );
```

sockfd : file descriptor of the socket (Welcome socket) at TCP server

return : On success, file descriptor for the accepted socket (connection socket)  
On error, -1 is returned





# Topic : Socket Programming with TCP



P<sub>1</sub>

TCP Client

socket()

connect()

P<sub>2</sub>

TCP Server

socket()

bind()

listen()

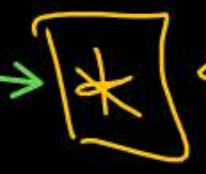
accept() → Conn. Socket

(connect return success)

(SYN) (Conn. Req.)

(SYN+ACK) (Accept)

FINAL-ACK





#Q. Identify the correct order in which a server process must invoke the function calls accept, bind, listen, and recv according to UNIX socket API.

[GATE-2015, Set-2, 1-Mark]

IIT-K

- ☒ (A) listen, accept, bind, recv
- ☒ (B) bind, listen, accept, recv
- ☒ (C) bind, accept, listen, recv
- ☒ (D) accept, listen, bind, recv

Ans: B

TCP server

- ① Socket
- ② Bind
- ③ Listen
- ④ Accept

#Q. A client process P needs to make a TCP connection to a server process S. Consider the following situation: the server process S executes a `socket()`, a `bind()` and a `listen()` system call in that order, following which it is preempted. Subsequently, the client process P executes a `socket()` system call followed by `connect()` system call to connect to the server process S. The server process has not executed any `accept()` system call. Which one of the following events could take place?

[GATE-2008]  
ISC, H.W.

- (A) `connect()` system call returns successfully
- (B) `connect()` system call blocks
- (C) `connect()` system call returns an error
- (D) `connect()` system call results in a core dump





## Topic : Write to a socket

#include <unistd.h>

```
ssize_t write ( int sockfd, const void buf[.count], size_t count);
```

Message

sockfd : file descriptor of the socket

return : On success, the number of bytes written is returned  
On error, -1 is returned





## Topic : Read from a socket

```
#include <unistd.h>
```

```
ssize_t read ( int sockfd, void buf[.count], size_t count);
```

sockfd : file descriptor of the socket

return : On success, the number of bytes read is returned  
On error, -1 is returned



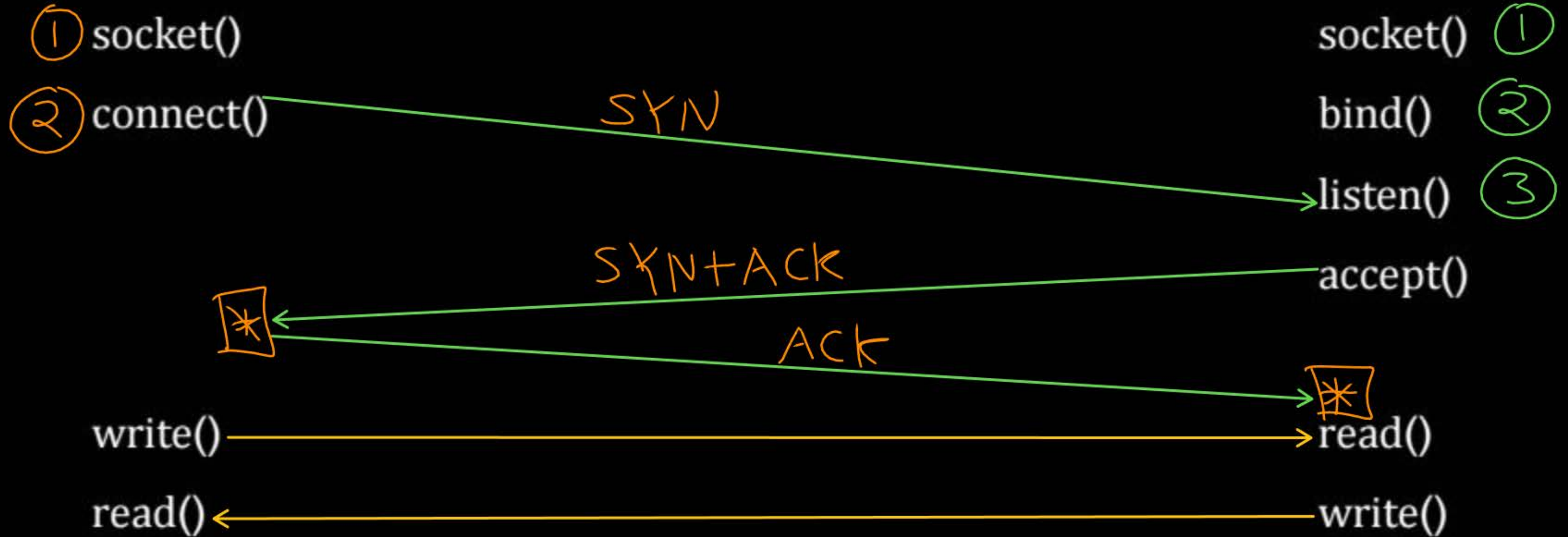
# Topic : Socket Programming with TCP



P<sub>2</sub>

TCP Client

TCP Server







## Topic : Close a socket



```
#include <unistd.h>
```

```
ssize_t close ( int sockfd );
```

→ send FIN segment

sockfd : file descriptor of the socket

return : On success, returns zero  
On error, -1 is returned



## Topic : Socket Programming with TCP



### TCP Client

socket()

connect()

write()

read()

close()

### TCP Server

socket()

bind()

listen()

accept()

read()

write()

close()

4-way handshake





## Topic : Server Type



→ Several clients can request service of the server in the same time.

### Non-concurrent Server :

- Service one client at one time
- Other client request must wait

### Concurrent Server :

- Services all client requests simultaneously





# Topic : Socket Programming with TCP

## Non-concurrent TCP Server :-

socket()

bind()

while(1) {

listen()

accept()

read() / write() }

close()



# Topic : Socket Programming with TCP



## Concurrent TCP Server :-

socket()

bind()

while(1) {

listen()

if ( fork() == 0 ) {

accept()

read() / write()

close() } }

close()



## Topic : Send a message on socket

```
#include <sys/socket.h>
```

```
ssize_t sendto ( int socket, const void *message, size_t length, int flags,  
                const struct sockaddr *dest_addr, socklen_t dest_len );
```

sockfd : file descriptor of the socket

return : On success, the number of bytes send is returned  
On error, -1 is returned





## Topic : Receive a message from a socket

```
#include <sys/socket.h>
```

```
ssize_t recvfrom ( int socket, void *buffer, size_t length, int flags,  
                  struct sockaddr *address, socklen_t address_len );
```

sockfd : file descriptor of the socket

return : On success, the number of bytes receive is returned  
On error, -1 is returned



# Topic : Socket Programming with UDP



P<sub>1</sub>

UDP Client

① socket()

sendto()

recvfrom()

close() ✓

P<sub>2</sub>

UDP Server ✓

socket() ①

bind() ②

recvfrom()

sendto()

close()





# Topic : Connected UDP sockets



P<sub>1</sub>

UDP Client

socket()

connect()

write()

read()

close()

connect

P<sub>2</sub>

UDP Server

socket()

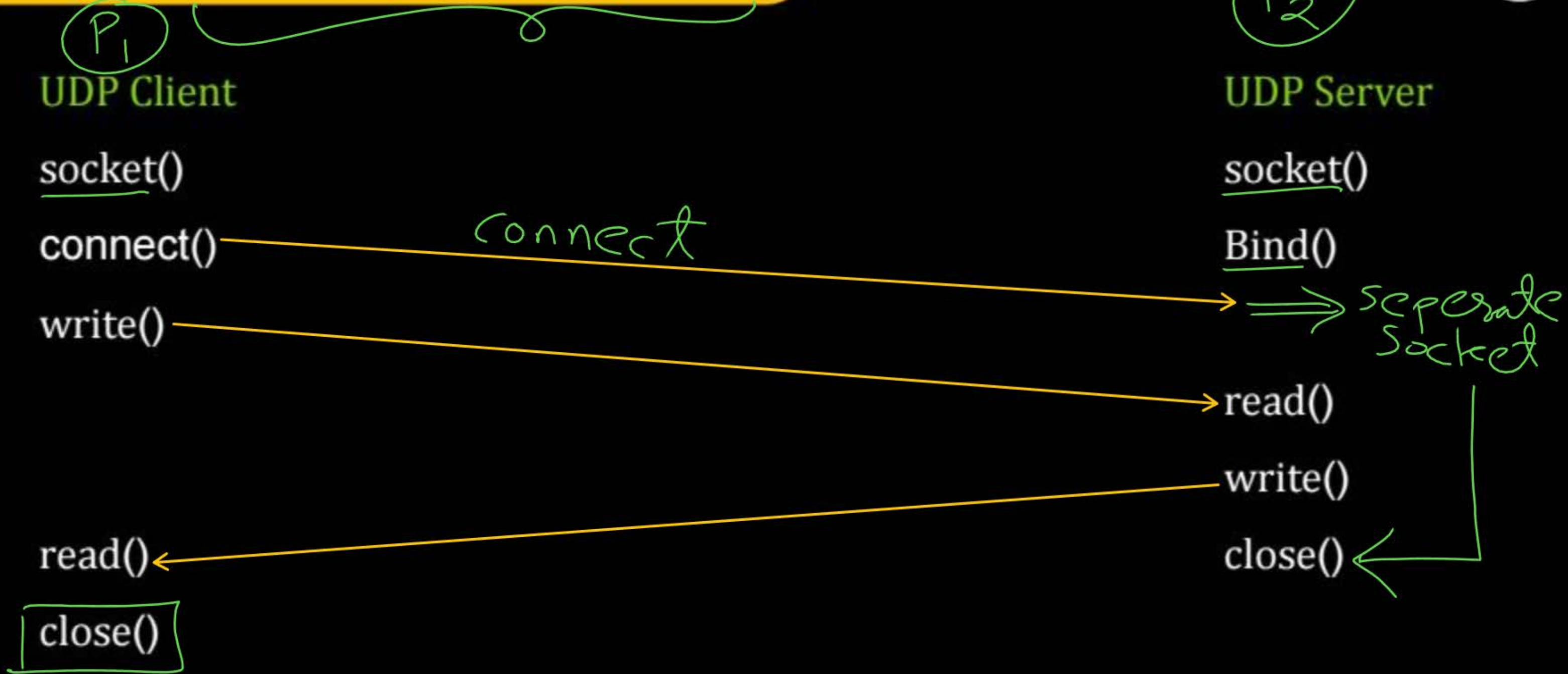
Bind()

read()

write()

close()

seperate  
socket





#Q. Consider socket API on a Linux machine that supports connected UDP sockets. A connected UDP socket is a UDP socket on which connect function has already been called. Which of the following statements is/are CORRECT?

- I. A connected UDP socket can be used to communicate with multiple peers simultaneously.
- II. A process can successfully call connect function again for an already connected UDP socket.

- (A) I only
- (B) II only
- (C) III Both I and II
- (D) Neither I nor II

[GATE-2017, Set-2]  
IIT-R, H.W.



**2 mins Summary**



**Topic**

**Socket Programming**



# THANK - YOU

